Fault Attacks on MPCitH Signature Schemes

Harrison Banda, Jan Brinkmann, and Juliane Krämer

Universität Regensburg, Regensburg, Germany,
firstname.lastname@ur.de

Keywords: MPCitH · FAEST · Mirath · MQOM · PERK · RYDE · SDitH · fault attacks

Abstract. In this work, we present two fault attacks against MPCitH-based signature schemes: we present a key-recovery attack and a signature-forgery attack, both of which only need a single successful fault injection to succeed. Focusing on Mirath and RYDE, we provide a detailed analysis of their behavior under the fault attacks presented. In addition, we evaluate the applicability of the same attacks to the other MPCitH-based candidates from round 2 of the NIST signature standardization process (FAEST, MQOM, PERK, and SDitH). Our analysis shows that all six schemes are vulnerable to at least one of the attacks. We validate the practicality of our attacks using the ChipWhisperer setup and discuss countermeasures to prevent the attacks.

1 Introduction

NIST's search for additional digital signature schemes within the post-quantum cryptography standardization process has brought a new family of post-quantum signature schemes into the focus of the research community: MPC-in-the-Head signatures, i.e., signature schemes which make use of multi-party computation (MPC). In the current second round of the process, six of these schemes are being analyzed: FAEST [7], Mirath [2], MQOM [9], PERK [1], RYDE [4], and SDitH [3]. Since both the MPC-in-the-Head (MPCitH) construction for digital signature schemes in general and the six schemes in particular are rather young, only a few results regarding their physical security exist yet.

A key distinction from other post-quantum signature schemes is that MPCitH-based signatures do not rely on structured algebraic trapdoors for their security. Instead, they rely on cryptographic primitives and zero-knowledge techniques, which simplifies implementation and offers potential robustness against algebraic attacks. MPCitH-based signatures benefit from compact public keys and relatively efficient verification procedures. However, signature sizes tend to be large, and signing can be expensive due to the need to simulate many MPC instances.

Concretely, a digital signature scheme is considered MPC-in-the-Head when it is constructed from a zero-knowledge proof realized through the MPC-in-the-Head paradigm [24]. In this paradigm, the prover simulates a multi-party computation protocol internally ("in the head"), commits to the virtual parties'

views, and selectively opens a subset of them for verification. To obtain a digital signature, a suitable one-way function, such as AES or the syndrome decoding function, is combined with the MPCitH construction, and the interactive proof is made non-interactive through the Fiat–Shamir transform [20].

In this sense, FAEST, although categorized by NIST as a symmetric-based signature scheme, is also an MPCitH scheme since it uses AES as its one-way function within the same proof structure. We therefore include FAEST in our analysis alongside the other MPCitH-based candidates, as it follows the same underlying paradigm and shares similar design principles, optimization strategies, and potential fault-attack surfaces.

Since MPCitH-based schemes inherently exhibit a nonzero soundness error, they are susceptible to the Kales–Zaverucha attack [28]. To reduce the probability of successful forgery, the underlying protocol must be repeated in multiple parallel rounds, which directly contributes to the large signature sizes observed in these schemes. To improve efficiency, cryptographic hash functions and GGM tree-based structures [22] are employed for share expansion and commitment generation, optimizing both proof size and verification cost.

To mitigate efficiency drawbacks, most recent schemes adopt advanced frameworks such as Threshold-Computation-in-the-Head (TCitH) [17,18] or VOLE-in-the-Head (VOLEitH) [8], where VOLE stands for vector oblivious linear evaluation. All six MPCitH candidates in the current second round of NIST's additional call for signature schemes make use of TCitH and/or include variants based on VOLEitH. PERK was the last of these to adopt VOLEitH, making the switch in its version 2.1 update, thereby joining FAEST and SDitH, which had already been based on VOLEitH. Despite differences in the underlying techniques, these schemes share structural features such as hashing and tree-based commitment strategies, which, while beneficial to performance, may expose them to physical security threats. Particularly fault attacks seem to be promising from an attacker's perspective, since MPCitH-based schemes rely on deterministic functions which lead to predictable patterns, and hence enable precisely timed faults that can lead to secret-key recovery or fault-enabled signature forgery. In the latter case, an adversary prepares an invalid signature and injects a fault into a deterministic verification operation so that the modified execution accepts the forgery. Throughout the paper, we use the term forgery to denote such fault-enabled signature forgery attacks, unless stated otherwise.

1.1 Related Work

Although MPCitH signature schemes are rather young, they have already attracted some research efforts in the field of physical security: Aranha et al. [5] initiated the research on physical security of MPCitH signature schemes by analyzing side-channel protections. They focused on the Picnic signature scheme, which was an alternate candidate in the third round of the initial post-quantum standardization process by NIST.

They proposed improved masking techniques for the underlying zero-knowledge proof system.

Gordad et al. [21] presented a single-trace side-channel attack targeting the MPCitH paradigm, specifically the TCitH framework. Their work reveals a side-channel leakage in the SDitH algorithm. By exploiting leakage from Galois field multiplication operations, the authors predict intermediate values and take advantage of the structure of the protocol to reconstruct the secret key. The attack is validated both in simulation and on real hardware, achieving full key recovery across all security levels. Most recently, Feneuil et al. [19] studied side-channel resistance of post-quantum signature schemes based on the TCitH framework. They analyzed side-channel leakage paths and introduced three novel tweaks to improve the masking-friendliness of both the Merkle and GGM tree variants of TCitH. They further discuss the applicability of their techniques to VOLEitH-based schemes.

In the field of fault attacks, Mondal et al. [30] analyzed fault injection attacks against zero-knowledge-based signature schemes, focusing specifically on the manipulation of the seed reveal function used in tree-based share expansion. Their work demonstrates that faults targeting seed expansion can lead to the recovery of all intermediate node values, including hidden leaf nodes, enabling full secret key extraction. This analysis is applied to the code-based schemes LESS and CROSS.

Given that MPCitH-based schemes are inherently built on zero-knowledge proofs, this attack model is relevant for such schemes as well. Jendral et al. [27] introduced an instruction-skipping attack on the SHAKE-256 hash function. The attack specifically targets the absorption phase of the hash function and enables secret-key recovery from a single fault injection. This technique has since been adapted to other schemes. For example, it has been extended in follow-up work to MAYO [26], where faults are used to compromise the vinegar seed generation process, again leading to key recovery. The successful transfer of the attack vector to MAYO demonstrates a weakness in hash-based constructions, which are commonly used in many post-quantum signature schemes. Recently, Jendral and Dubrova [25] presented the most comprehensive fault and side-channel analysis to date on a VOLEitH based signature scheme. Using FAEST as a case study, they introduce several attack vectors, including an attack on AES in counter mode to manipulate the generation of child nodes within the tree structure. The authors also generalize their methodology to schemes using similar constructions, such as those based on MPCitH, arguing that the structural similarities make these schemes vulnerable despite implementation differences. However, we show that the said attacks do not work against other MPCitH schemes, since only FAEST uses the AES counter mode for seed creation. In addition to these vectors, Jendral and Dubrova also describe a fourth attack, referred to as the VOLE conversion abort attack, which targets the transformation of the initial seeds within the GGM tree during the VOLE commitment phase and exploits the conversion process to induce aborts and extract information. The authors argue that this attack, although originally studied in the VOLEitH setting, may also be applicable to related constructions, with the SDitH specification being referred to as an example.

Most recently, Sarde and Debande [31] presented the first fault attacks on MQOM. They introduce four differential fault attacks targeting the MQ evaluation phase. Two attacks recover the entire secret key through linear algebra and using one or two random faults, while two others target the MQ system coefficients directly. Their work shows that these attacks remain effective even against masked implementations, highlighting new vulnerabilities in MPCitH-based constructions.

1.2 Contribution

We set out to better understand the physical security of MPC-in-the-Head signature schemes and present a detailed evaluation of fault attacks on RYDE [4] and Mirath [2]. We further assess the applicability of these attacks to the remaining MPCitH-based candidates, all of which are part of the ongoing round 2 of NIST's standardization process for digital signature schemes: FAEST [7], MQOM [9], PERK [1], and SDitH [3].

We present two fault attacks, a key-recovery attack and a signature-forgery attack, both exploiting the structural properties shared among MPCitH-based schemes:

1. The first attack, the key-recovery attack, targets the seed generation process in the GGM tree through fault injection. By forcing intermediate sibling nodes to be identical, the structure of the tree collapses, making it possible to recover all leaf values, and ultimately enabling full secret-key recovery with a single successful fault injection. We identified several attack vectors to achieve this effect. Two of them exploit the fact that the tree nodes are initialized to zero and then skip either the tree expansion or the seed expansion function responsible for computing the node values. The third attack vector ensures that two child nodes obtain the same value by skipping the XOR instruction that is intended to differentiate the sibling nodes.
2. The second attack, the signature-forgery attack, targets the challenge of a linear combination in the Fiat–Shamir transform by manipulating the challenge, often the matrix $\mathbf{\Gamma}$. By applying a fault during challenge generation within the verification process, the attacker can fix $\mathbf{\Gamma}$, enabling the acceptance of forged signatures via a Kales–Zaverucha-like attack.

Based on the updated version 2.1 specifications released in September 2025, we analyze all six schemes to determine whether they are vulnerable to the attacks presented in this work. In theory, all six schemes admit both key-recovery and signature-forgery attacks. For Mirath and RYDE, we also practically executed all attacks presented in this work. We chose their reference implementations as target implementations, since embedded implementations are not yet available. Although we do not analyze optimized implementations, our results identify critical issues that must be addressed in hardened versions of these schemes. We describe the experimental setting used to execute these attacks in practice and discuss appropriate countermeasures. Our results are summarized in

Table 1. We further assess the transferability of attack techniques from related work to MPC-in-the-Head signature schemes and show that most previously proposed attack vectors are not applicable (see Section 3.4).

Signature Scheme	Key-Recovery Attack			Forgery Attack
	tree	**seed**	**XOR**	
Mirath	✓	✓	✓	✓
RYDE	✓	✓	✓	✓
SDitH	∘	∘	(✓)	∘
MQOM	∘	∘	×	(✓)
PERK	(✓)	(✓)	(✓)[1]	(✓)
FAEST	(✓)	(✓)	×	∘

Table 1. Overview of fault injection attacks on MPCitH signature schemes presented in this work. The key-recovery attack is categorized into three attack vectors: tree generation (**tree**), seed generation (**seed**), and XOR instruction (**XOR**). A checkmark symbol, ✓, indicates a theoretically feasible attack that we also practically implemented, (✓) denotes a theoretically feasible attack with an identified target in the reference implementation, ∘ represents a theoretically feasible attack without known target, and × marks infeasible attacks.

1.3 Organization

In Section 2, we describe the background necessary to understand the analyzed schemes and the attacks against them. We present the key-recovery attack and the signature-forgery attack, together with countermeasures to prevent them, in Sections 3 and 4, respectively. Here, we show in detail how they can be applied to Mirath and RYDE. In Section 5, we analyze the applicability of the two fault attacks to the other MPCitH signature schemes, namely FAEST, MQOM, PERK, and SDitH. In Section 6, we describe the ChipWhisperer setup we used for the execution of the attacks and demonstrate the feasibility of the attacks by successfully executing them on Mirath and RYDE. We conclude in Section 7.

2 Background

This section introduces the cryptographic foundations necessary to understand the MPCitH signature schemes analyzed in this work. We focus primarily on RYDE and Mirath, schemes built upon hard problems in the rank metric and instantiated using the TCitH-PIOP framework.

[1] The XOR attack vector in PERK is only feasible if AES is used for node creation.

Section 2.1 provides an overview of the GGM tree structure and the BAVC technique, which are employed to optimize the share-opening process in the analyzed signature schemes. Section 2.2 outlines the MPCitH framework, encompassing both the VOLEitH and TCitH-PIOP formalisms that underpin the analyzed schemes and form the basis of our attacks. Finally, Section 2.3 describes the structure of the considered MPCitH schemes, with particular focus on RYDE and Mirath.

2.1 GGM Trees and BAVC

All six MPCitH signature schemes considered in our analysis use GGM trees to distribute secret shares among multiple virtual parties [29]. GGM trees are binary trees, where each node has at most two child nodes, and the structure is generated top-down starting from a root seed. In the context of MPC-in-the-Head protocols, GGM trees are used to commit to additive secret sharings using an all-but-one vector commitment (AVC) scheme, where for a secret split into N shares, $N-1$ are opened. GGM trees allow for efficient representation of shares, since an opening path can be provided instead of revealing all $N-1$ shares individually. This path consists of all nodes required to recompute the $N-1$ leaves, and whenever both children of a parent node are known, they can be replaced by the parent node itself, reducing the number of required nodes in the opening. This path is included in the signature.

To reduce the probability of a successful forgery, the protocols repeat the commitment procedure τ times, corresponding to τ independent parallel iterations of the protocol. The schemes FAEST, Mirath, PERK, RYDE, and SDitH implement the so-called one-tree optimization [6], which enables a batched all-but-one vector commitment (BAVC) approach.

Instead of constructing τ independent GGM trees, a single large tree is generated to commit to all iterations simultaneously. The leaves are organized such that, for each iteration index $e \in \{1, 2, \ldots, \tau\}$, the first τ leaf nodes correspond to the first share of iteration e. For example, the leaf with index τ represents the second share of the first iteration ($e = 1$).

This design improves efficiency because, with high probability, some of the authentication paths associated with unopened shares overlap near the leaves. In this context, we distinguish between hidden nodes, which are not revealed during verification, and revealed nodes, which belong to an opening path that enables the verifier to authenticate a specific leaf from the root commitment.

MQOM further employs the correlated tree optimization [23], which reduces the computational cost of tree generation. In this approach, only one side of each sibling pair is generated using a pseudorandom generator (PRG), and the corresponding sibling is derived using an XOR operation with the parent node, effectively halving the number of PRG calls required.

2.2 MPC-in-the-Head Framework Overview

The MPC-in-the-Head paradigm, introduced by Ishai et al. [24], provides a framework for constructing zero-knowledge proofs from secure multi-party computation protocols. In this approach, the prover simulates an N-party MPC protocol "in their head" where the secret witness is secret-shared among the parties, commits to the views of all virtual parties, receives challenges from the verifier to open a subset of these views, and finally reveals the requested views along with consistency proofs. The key insight is that if the MPC protocol correctly computes the desired function and can tolerate a certain number of corrupt parties, then revealing a random subset of views allows the verifier to check correctness while learning nothing about the secret witness.

The security of MPCitH-based schemes is based on the completeness of the underlying MPC protocol, the privacy against coalitions of up to t corrupt parties, and the binding and hiding properties of the commitment scheme.

A signature scheme is considered MPCitH if it follows the core paradigm of simulating a multi-party computation protocol in the prover's head, committing to the virtual parties' views, and selectively opening subsets for verification. In our analysis, we examine FAEST, Mirath, MQOM, PERK, RYDE, and SDitH, as all of them implement this MPCitH approach. Although NIST classifies FAEST separately as a symmetric-based signature, we include it in our MPCitH analysis because it fundamentally adheres to the same MPC-in-the-Head paradigm.

Recent MPCitH frameworks have evolved to optimize for specific applications, particularly digital signatures, leading to two prominent approaches that will be discussed in the following subsections.

TCitH and the PIOP Framework Most MPCitH schemes use the TCitH paradigm [18,17], which is formalized as a Polynomial Interactive Oracle Proof (PIOP) [16]. In this model, the prover commits to a set of polynomials whose coefficients encode the secret witness. The verifier then issues a random linear combination challenge, followed by a request to evaluate the committed polynomials at a randomly chosen point.

Specifically, suppose the goal is to prove knowledge of a witness w satisfying polynomial constraints $\{f_j(w) = 0\}$. In the PIOP approach, the prover samples random polynomials $P_i(X) = w_i\, X + (w_{\texttt{base}})_i, \quad \texttt{for } i = 1, \ldots, n$, along with an additional polynomial $P_0(X)$ of degree $(d-1)$, where d is the degree of constraints. These polynomials are committed to. The verifier then sends a challenge consisting of random coefficients $\gamma_1, \ldots, \gamma_l$ from an extension field. The prover responds with polynomial

$$Q(X) = P_0(X) + \sum_{j=1}^{l} \gamma_j f_j^{[h]}(X, P_1(X), \ldots, P_n(X)),$$

where each $f_j^{[h]}$ is the homogenized version of f_j. Finally, the verifier selects a random evaluation point z from a public set S and requests evaluations $P_i(z)$ and

$Q(z)$, along with consistency proofs. Soundness is guaranteed because, with high probability, a cheating prover cannot make $Q(z)$ satisfy the expected constraints if the witness is invalid.

VOLE-in-the-Head Framework The VOLE-in-the-Head paradigm [8] is another powerful framework for constructing efficient zero-knowledge proofs. Instead of committing to polynomials, VOLEitH relies on information-theoretically secure commitments built from Vector Oblivious Linear Evaluation (VOLE) correlations.

In this model, the prover's witness is encoded in a set of linear polynomials over a binary extension field. The core building block is a VOLE correlation: for a length $\hat{\ell}$, the prover holds $(\mathbf{u}, \mathbf{v}) \in \mathbb{F}_2^{\hat{\ell}} \times \mathbb{F}_{2^m}^{\hat{\ell}}$, and the verifier holds $(\mathbf{g}, \Delta) \in \mathbb{F}_{2^m}^{\hat{\ell}} \times \mathbb{F}_{2^m}$, satisfying the linear relation

$$g_i = u_i \cdot \Delta + v_i \qquad \text{for } i \in [0, 1, \ldots, \hat{\ell} - 1].$$

This structure forms a linearly homomorphic commitment to the prover's bits $\mathbf{u}$.

The protocol flow in VOLEitH begins with the prover generating multiple VOLE correlations, one for each protocol repetition. Let τ denote the number of parallel instances executed in a single proof. For each $e \in [\tau]$, the prover samples a VOLE pair $(\mathbf{u}_e, \mathbf{v}_e)$ and commits to it using an all-but-one vector commitment scheme, typically instantiated via a GGM tree. The prover then embeds the secret witness into these VOLE correlations and make use of their linear homomorphic properties to compute proofs of witness correctness.

Next, the verifier issues a challenge Δ, prompting the prover to combine the τ small-field VOLE instances into a single large-field instance. In this step, the verifier also provides random coefficients defining a linear combination of the constraints to be verified. Finally, during the opening and verification phase, the prover opens the relevant commitments and sends a masked version of the combined linear relation. The verifier reconstructs the VOLE correlations from the opened commitments and checks that the relation holds, thereby confirming that the prover's computations are consistent with a valid witness.

Soundness is achieved through the binding property of the VOLE correlations and the randomness of the verifier's challenges. The prover's ability to successfully open the commitments and pass the linear verification check implies, with high probability, that they possess a valid witness. This framework has been successfully instantiated in several post-quantum signature schemes, including PERK (since v2.1), FAEST, and SDitH, and also in the variants of RYDE and Mirath.

2.3 MPCitH Signature Schemes

This subsection provides an overview of the MPCitH-based signature schemes submitted to round 2 of NIST's additional call for signatures. We describe RYDE

in detail, Mirath follows naturally because of its structural similarity. For the remaining candidates; FAEST, MQOM, PERK and SDitH, we refer the reader to their respective specifications [7,9,1,3] for further details.

Overview of RYDE and Mirath RYDE and Mirath share a common structure: both are constructed using the TCitH framework and derive their security from rank-metric problems. RYDE is based on the rank syndrome decoding (RSD) problem [4, Definition 3], whereas Mirath builds on the MinRank problem [13], in particular, its syndrome decoding variant [2, Definition 2]. Both schemes use dual support decomposition [10] to prove knowledge of a low-rank witness.

Dual Support Modeling Let $\mathbb{F}_{q^m}$ be a finite field extension of $\mathbb{F}_q$, and let $\mathbf{H} \in \mathbb{F}_{q^m}^{(n-k)\times n}$ be a parity-check matrix of an $[n,k]$ linear code over $\mathbb{F}_{q^m}$. A secret vector $\mathbf{x} \in \mathbb{F}_{q^m}^n$ of rank r is decomposed using dual support modeling as $\mathbf{x} = \mathbf{s}\,\mathbf{C}$, where $\mathbf{s} = (1 \parallel \mathbf{s}') \in \mathbb{F}_{q^m}^r$ with $\mathbf{s}' \in \mathbb{F}_{q^m}^{r-1}$ and $\mathbf{C} = [\mathbf{I}_r \parallel \mathbf{C}'] \in \mathbb{F}_q^{r\times n}$ with $\mathbf{C}' \in \mathbb{F}_q^{r\times(n-r)}$. The rank syndrome decoding constraint $\mathbf{x}\mathbf{H}^\top = \mathbf{y}$, `with syndrome` $\mathbf{y} \in \mathbb{F}_{q^m}^{n-k}$, becomes the quadratic constraint $\mathbf{s}\,\mathbf{C}\,\mathbf{H}^\top = \mathbf{y}$. The witness is $(\mathbf{s}', \mathbf{C}')$, and the protocol proves knowledge of this witness satisfying the constraint.

In Mirath, the error matrix $\mathbf{E} \in \mathbb{F}_q^{m\times n}$ is decomposed as $\mathbf{E} = \mathbf{S}\,\mathbf{C}$, with $\mathbf{S} \in \mathbb{F}_q^{m\times r}$, $\mathbf{C} \in \mathbb{F}_q^{r\times n}$, `and` $\mathrm{rank}(\mathbf{E}) = r$. Here again, $\mathbf{C}$ is fixed as $[\mathbf{I}_r \parallel \mathbf{C}']$, and the goal is to find $\mathbf{S}$ and $\mathbf{C}'$. The associated constraint becomes $\mathbf{H}\,\mathrm{vec}(\mathbf{S}\,\mathbf{C}) = \mathbf{y}$, where $\mathrm{vec}(\cdot)$ returns a flattened version of the input matrix.

Mirath's interactive proof is similar to that of RYDE. For this reason, we focus on RYDE, and the description extends naturally to Mirath.

RYDE's Interactive Proof RYDE instantiates the TCitH framework for the RSD problem. For clarity, we summarize the key protocol parameters used. The symbol ρ denotes the number of linear combinations in the first challenge, while τ represents the number of parallel repetitions of the protocol. The parameter N specifies the size of the evaluation set $S \subset \mathbb{F}_{q^\mu}$. The threshold $T_{\mathtt{open}}$ defines the maximum size of openings in the BAVC procedure, and finally, w is the grinding security parameter that determines the expected number of hash queries required to find a valid challenge.

The interactive protocol proceeds as follows:

1. Commitment Phase: For each repetition $e \in \{1, \ldots, \tau\}$
   - Sample degree-1 polynomials:

     $$P_{\mathbf{s}'}(X) = \mathbf{s}'\cdot X + \mathbf{s}'_{\text{base}}, \quad P_{\mathbf{C}'}(X) = \mathbf{C}'\cdot X + \mathbf{C}'_{\text{base}}, \quad P_{\mathbf{v}}(X) = \mathbf{v}\cdot X + \mathbf{v}_{\text{base}}$$

   - Commit using batched all-but-one vector commitment (BAVC) with GGM tree:
     - Generate N seeds $\{\mathsf{seed}_i\}_{i=1}^N$ via GGM tree

- Expand each seed to $(\mathbf{s}'_{\mathsf{rnd},i}, \mathbf{C}'_{\mathsf{rnd},i}, \mathbf{v}_{\mathsf{rnd},i})$ using PRG
- Compute accumulated values:

$$\mathbf{s}'_{\mathsf{acc}} = \sum_{i=1}^{N} \mathbf{s}'_{\mathsf{rnd},i}, \quad \mathbf{C}'_{\mathsf{acc}} = \sum_{i=1}^{N} \mathbf{C}'_{\mathsf{rnd},i}, \quad \mathbf{v}_{\mathsf{acc}} = \sum_{i=1}^{N} \mathbf{v}_{\mathsf{rnd},i}$$

$$\mathbf{s}'_{\mathsf{base}} = -\sum_{i=1}^{N} \phi(i) \cdot \mathbf{s}'_{\mathsf{rnd},i}, \quad \mathbf{C}'_{\mathsf{base}} = -\sum_{i=1}^{N} \phi(i) \cdot \mathbf{C}'_{\mathsf{rnd},i}$$

and

$$\mathbf{v}_{\mathsf{base}} = -\sum_{i=1}^{N} \phi(i) \cdot \mathbf{v}_{\mathsf{rnd},i}$$

- Reveal auxiliary values:

$$\mathbf{s}'_{\mathsf{aux}} = \mathbf{s}' - \mathbf{s}'_{\mathsf{acc}}, \quad \mathbf{C}'_{\mathsf{aux}} = \mathbf{C}' - \mathbf{C}'_{\mathsf{acc}}$$

2. First Challenge: The verifier sends random matrix $\mathbf{\Gamma} \in \mathbb{F}_{q^m}^{(n-k)\times\rho}$.
3. Response Polynomial: The prover computes
   $P_\alpha(X) = P_{\mathbf{v}}(X) + \mathbf{\Gamma} \cdot \left(\mathbf{H} P_{\mathbf{x}}(X) - \mathbf{y}X^2\right)$, where
   $P_{\mathbf{x}}(X) = (X \parallel P_{\mathbf{s}'}(X)) \cdot [\mathbf{I}_r X \parallel P_{\mathbf{C}'}(X)]$.
   If we let $P_{\mathbf{x}}(X) = \mathbf{x}X^2 + \mathbf{x}_{\mathtt{mid}}X + \mathbf{x}_{\mathtt{base}}$, and $\mathbf{x} = [\mathbf{s} \parallel \mathbf{s}\mathbf{C}']$,
   $\mathbf{x}_{\mathtt{mid}} = [\mathbf{s}_{\mathtt{base}} \parallel \mathbf{s}_{\mathtt{base}}\mathbf{C}' + \mathbf{s}\mathbf{C}'_{\mathtt{base}}]$ and $\mathbf{x}_{\mathtt{base}} = [\mathbf{0} \parallel \mathbf{s}_{\mathtt{base}}\mathbf{C}'_{\mathtt{base}}]$,
   where $\mathbf{s} = (1 \parallel \mathbf{s}')$ and $\mathbf{s}_{\mathtt{base}} = (1 \parallel \mathbf{s}'_{\mathtt{base}})$. Then $P_\alpha(X) = \alpha_{\mathtt{mid}}X + \alpha_{\mathtt{base}}$,
   where $\alpha_{\mathtt{mid}} = \mathbf{\Gamma} \cdot \mathbf{x}_{\mathtt{mid}}\mathbf{H}^\top + \mathbf{v}$ and $\alpha_{\mathtt{base}} = \mathbf{\Gamma} \cdot \mathbf{x}_{\mathtt{base}}\mathbf{H}^\top + \mathbf{v}_{\mathtt{base}}$.
4. Second Challenge: The verifier sends random evaluation point $z \in S \subset \mathbb{F}_{q^m}$.
5. Open Evaluations: The prover reveals $\mathbf{s}'_{\mathtt{eval}} = P_{\mathbf{s}'}(z)$, $\mathbf{C}'_{\mathtt{eval}} = P_{\mathbf{C}'}(z)$ and $\mathbf{v}_{\mathtt{eval}} = P_{\mathbf{v}}(z)$ with BAVC opening proof π for all seeds except seed_{i^*} where $z = \phi(i^*)$.
6. Verification: The verifier computes $\mathbf{x}_{\mathtt{eval}} = (z \parallel \mathbf{s}'_{\mathtt{eval}}) \cdot [z\mathbf{I}_r \parallel \mathbf{C}'_{\mathtt{eval}}]$ and $\alpha_{\mathtt{eval}} = \mathbf{v}_{\mathtt{eval}} + \mathbf{\Gamma} \cdot \left(\mathbf{H}\mathbf{x}_{\mathtt{eval}}^\top - \mathbf{y}z^2\right)$, and checks $\alpha_{\mathtt{eval}} = \alpha_{\mathtt{mid}}z + \alpha_{\mathtt{base}}$ while validating the BAVC opening proof.

Non-Interactive Signature via Fiat-Shamir The non-interactive signature is obtained by replacing verifier challenges with hash outputs
$h_1 = \mathsf{Hash}_1(\mathsf{salt}, \{\mathsf{com}_{e,i}\}, \{\mathbf{s}'_{\mathsf{aux}}, \mathbf{C}'_{\mathsf{aux}}\})$ derives $\mathbf{\Gamma}$ and
$h_2 = \mathsf{Hash}_2(\mathsf{Hash}_0(\mathsf{msg}), \mathsf{pk}, \mathsf{salt}, h_1, \{\alpha_{\mathtt{mid}}, \alpha_{\mathtt{base}}\})$ derives $\{i^*(e)\}$.

Grinding: Iterate with counter ctr until $v_{\mathtt{grinding}} = 0$ and BAVC opening has $\leq T_{\mathtt{open}}$ revealed nodes.

Signature:

$$\sigma = (\mathsf{salt} \parallel \mathsf{ctr} \parallel h_2 \parallel \pi_{\mathtt{BAVC}} \parallel \{\mathbf{s}'_{\mathsf{aux}}, \mathbf{C}'_{\mathsf{aux}}, \alpha_{\mathtt{mid}}\}_{e=1}^{\tau}).$$

Verification: Recompute evaluations from BAVC opening, then verify h_2 matches recomputed hash.

The security of RYDE relies on the hardness of the RSD problem, the soundness of the TCitH proof system, and the security of the Fiat-Shamir transformation.

The parameters (ρ, τ, N, w) are carefully chosen to ensure that the generic forgery attack against five-pass Fiat-Shamir signatures by Kales and Zaverucha [28] remains computationally infeasible. As analyzed in the RYDE specification [4, Section 8.1], the attack complexity is given by

$$\text{cost}_{\texttt{forge}} = \min_{0 \le \tau' \le \tau} \left\{ \frac{1}{\sum_{i=\tau'}^{\tau} \binom{\tau}{i} p^i (1-p)^{\tau - i}} + \left(\frac{N}{2} \right)^{\tau - \tau'} \right\}$$

where $p = \frac{1}{q^{m\rho}}$. For all RYDE parameter sets, this cost exceeds 2^λ for the target security level λ, ensuring the attack is infeasible.

3 Key-Recovery Attack on GGM Tree Seed Generation

Recent research has exposed the susceptibility of zero-knowledge signature schemes to fault attacks by targeting their internal seed expansion processes [25,30], particularly when these schemes rely on tree-based structures such as the GGM tree. Our work builds on these insights and shows that the MPCitH-based signature schemes like Mirath and RYDE are vulnerable under this fault model. These schemes in general, rely on a tree structure with $\tau \cdot N$ leaves, which, while optimizing signature size, also increases the attack surface. Each round reveals all but one leaf seed, along with the opening paths required for verification. We show how faults in the seed generation process can reveal the hidden leaf, thereby enabling a secret key recovery. In this section, we identify three attack vectors—namely the expand-tree, expand-seed, and XOR attack vector—that exploit the seed-tree generation to recover the secret key in the Mirath and RYDE signature schemes.

3.1 Attacker Model

We assume an attacker with physical access to the signing device during legitimate signing operations. The adversary is able to induce faults at specific stages of the tree-based seed generation process, particularly during the computation of child seeds from a parent node in the GGM tree. The attacker also has access to the resulting (faulty) signature and the corresponding public information, including revealed opening paths. The goal of an attacker is to use this information to recover the hidden leaf and ultimately the secret key.

3.2 Attack Strategy

For each signature, a GGM tree is generated that realizes an AVC in which all but one secret share (in the form of a leaf node) are revealed. Our key-reveal attack exploits the fact that, with a successful fault injection, it is possible to recover

all shares, including the hidden one, in order to recover the complete secret. In our attack strategy, the fault is injected during the tree generation phase and the hidden leaves are selected afterwards during the signing procedure, with the selected leaves containing the information we aim to recover. We identify three potential attack vectors to achieve this.

With the expand-tree attack vector, we target the execution of the complete GGM tree generation. This approach requires that all tree nodes are initialized with known values. If the tree expansion operation is skipped, the nodes do not receive updated values and remain at their initialized state, meaning all nodes of an instance become known.

With the expand-seed attack vector we aim to disrupt the execution of node creation, in which two child nodes are derived from a parent node. Here again, initialization of the nodes with known values is required. Moreover, at least one descendant node must later be selected as a hidden node. Only in this case can a fault attack yield the information necessary for a key-reveal attack.

With the XOR attack vector, we focus on the node computation so that both child nodes derived from a parent node become identical. In many schemes, nodes are generated using AES from the parent node. For Mirath and RYDE the following computation is applied:

$$\mathrm{AES}_k\big(\mathrm{salt} \oplus (\mathrm{bin}_{\lambda-40}(a) \,\|\, \mathrm{bin}_{32}(\mathrm{idx}) \,\|\, \mathrm{bin}_8(0))\big)$$

$$\mathrm{AES}_k\big(\mathrm{salt} \oplus (\mathrm{bin}_{\lambda-40}(a) \,\|\, \mathrm{bin}_{32}(\mathrm{idx}) \,\|\, \mathrm{bin}_8(1))\big)$$

where k is derived from the parent node and the salt is freshly generated for each signature and included in the signature itself. To ensure that both child nodes have the same value, we can skip the XOR instruction that leads to the two nodes being different. For a successful key-reveal, it is necessary that at least one descendant node is later chosen as a hidden node, and that one of the two generated nodes is known. In this case, the sibling node can be inferred to take the same value, allowing the recovery of hidden information.

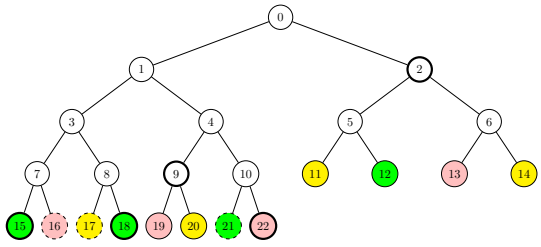

Fig. 1. Example of a faulted batched GGM tree with 12 leaves. Nodes of the same color belong to the same instance. Dashed borders are hidden nodes, while thick borders represent revealed nodes.

As an example, consider a GGM tree with 12 leaves, where $\tau = 3$ and $N = 4$. This means the AVC is executed three times with four secret shares each. In Figure 1, a hidden node is selected for each instance. Nodes belonging to the same instance are color-coded, e.g., nodes 12, 15, 18, and 21 belong to instance $e = 2$. A successful attack for the XOR attack vector could occur during the generation of nodes 9 and 10, since node 9 lies on the opening path and, together with node 21, can be used to recover the secret for an instance.

Recovering the Secret After Successful Fault As noted earlier, our detailed analysis focuses on the RYDE and Mirath schemes, which both rely on the TCitH. These schemes share a similar proof structure for demonstrating knowledge of the witness, meaning that once an attacker successfully recovers the hidden seed from the GGM tree in a particular round, the subsequent process for reconstructing the secret is largely identical in both cases.

We illustrate the recovery procedure concretely using RYDE as a representative example; the same reasoning applies directly to Mirath due to their shared structural design.

Once the attacker recovers a hidden seed, they apply the known signature function `ExpandShare` to each seed and obtain the shares $(\mathbf{s}'_{\text{rnd},i}, \mathbf{C}'_{\text{rnd},i})$. Since $\mathbf{s}'_{\text{aux}}$ and $\mathbf{C}'_{\text{aux}}$ are included in the signature, the adversary can directly solve the following equations to recover the secret $\mathbf{s}'$ and $\mathbf{C}'$:

$$\mathbf{s}' = \mathbf{s}'_{\text{aux}} + \sum_i \mathbf{s}'_{\text{rnd},i} \quad \texttt{and} \quad \mathbf{C}' = \mathbf{C}'_{\text{aux}} + \sum_i \mathbf{C}'_{\text{rnd},i}.$$

3.3 Practical Attack

In this subsection, we will discuss three attack vectors we have identified to attack the round 2 implementations of the MPCitH-based submissions with the attack strategy explained above. In the following, we evaluate the feasibility of the attacks for each signature scheme and describe how they can be applied to the reference implementation if necessary.

In the reference implementations of the round 2 submissions of Mirath and RYDE, both use the same `ggm_tree_expand` function. Therefore, the attacks described here apply to both schemes. Without loss of generality, we focus on the concrete functions of Mirath. The function `ggm_tree_expand` is located in the file `mirath_ggm_tree.c`. In RYDE, the equivalent function `ryde_3f_ggm_tree_expand` is defined in `ggm_tree.c`.

Skip the `mirath_ggm_tree_expand` function call: Before the program calls the function `ggm_tree_expand`, it initializes all seeds of the GGM tree with the value 0. We can therefore skip the function entirely, causing all values to remain at 0.

Skip a `mirath_expand_seed` function call: In this attack, a call to the function `mirath_expand_seed` is skipped. As a result, the lower-level nodes that would normally be created by this function remain at their initialized value of zero.

There are several possible function calls that can be attacked for this scenario. It is sufficient to skip a single `mirath_expand_seed` call assuming it is responsible for creating a subtree that contains at least one hidden leaf node.

Mirath and RYDE utilize a batched GGM tree structure, as outlined in Section 2.1, where the path used for the opening varies in each signature due to the random selection of hidden nodes. This randomness prevents most of the single nodes from being statically identified as a guaranteed target for a successful key recovery. To better understand which nodes are most vulnerable to fault injection, we developed a simulation tool that estimates the success probability of recovering the secret key when skipping a specific node. The tool generates batched GGM trees with randomly selected hidden nodes and computes the success probability at each level. Using parameters from `Mirath-1a-fast` with $N = 256$, $\tau = 17$, and $T_{open} = 118$, we performed 10,000 simulations. As expected, the first two levels of the tree always lead to successful key recovery due to their structural inclusion in every opening path. Beyond that, the probability declines: at level 2 it drops to 97.43 %, and at level 3 to 81.88 %, with decreasing probabilities at subsequent levels.

Skip the XOR Operation in `aes_128_expand_seed`: In this attack, the exploitation does not rely on initialized zero values. Instead, we force two sibling nodes to become identical by skipping a specific XOR instruction during their generation. Both Mirath and RYDE generate tree nodes using AES, where the encryption key is derived from the parent node. For the message input, the implementation XORs a previously generated salt with a fixed mask. The only difference between the two sibling nodes lies in the first byte of the mask: the left node uses `0x00`, while the right node uses `0x01` (see Listing 1.1). In the reference implementation, both AES calls operate on the same base message `msg`, which is modified in a single instruction. If an attacker successfully skips the XOR operation at line 14, the resulting left and right child nodes become identical. We further believe that the same effect could be achieved through a targeted bit-flip. However, we did not investigate this alternative fault model further in this work.

In Mirath and RYDE, not every skipped XOR operation in the `aes_128_expand_seed` function results in a successful key recovery. For such an attack to succeed, the targeted seed creation must produce one node that is part of the opening path and another node that is not. This condition ensures that, by knowing the value of the node included in the opening, it is possible to compute the subnodes of the hidden sibling node, which finally leads to a key recovery.

```
..
memcpy(msg, salt, sizeof(uint8_t) * 16);
msg[0] ^= 0x00;
for (size_t k = 0; k < 4; k++) {
    msg[k + 1] ^= ((uint8_t *)&idx)[k];
}
msg[5] ^= domain_separator;
aes_128_encrypt(&key, msg, dst[0]);
msg[0] ^= 0x01;
aes_128_encrypt(&key, msg, dst[1]);
...
```

Listing 1.1. Generation of two seeds in the `aes_128_expand_seed` function.

To extend our analysis from the previous attack vector, we used the same simulation tool to evaluate the practical effectiveness of fault attacks under the parameter set of `Mirath-1a-fast`, specifically with $N = 256$, $\tau = 17$, and $T_{open} = 118$. The tool generates batched GGM trees with randomly selected hidden nodes and evaluates the probability of successful key recovery depending on the targeted node. In contrast to the scenario described for the last attack vector, we define a successful attack here as one in which at least one of the affected nodes is revealed and at least one of them is responsible for generating a hidden node. This tool allows us to estimate the most promising fault injection targets in realistic settings and to quantify their recovery potential based on tree level.

The highest recovery probability was observed at level 3, corresponding to nodes 7 through 14, with a success rate of 47.94 %, followed by level 4 with a probability of 43.18 %. This implies that, for instance, an attack on node 7, which is responsible for generating nodes 15 and 16, will result in a successful key recovery in roughly two signature generations.

3.4 Failed Attack Vectors

While evaluating fault attack resilience of MPCitH-based schemes, we considered several attack vectors that have proven effective in other post-quantum signature schemes. However, we found that these strategies could not be successfully applied to the current round 2 versions of the schemes we analyzed in this submission. This subsection briefly explains why these attack ideas are not transferable.

SHAKE Glitch on Seed Expansion Earlier versions of the candidate schemes used SHAKE to expand seeds in their GGM tree construction. This opened them to fault injection attacks targeting the SHAKE function. A glitch on SHAKE, such as the one demonstrated in [27] by Jendral et al., could have allowed manipulation of internal outputs, facilitating full seed recovery. However, in the round 2 specifications, SHAKE has been replaced by AES for seed expansion. This change renders the attack infeasible, as the previous fault model, targeting SHAKE, no longer applies.

Instruction Glitch During Seed Reveal Another proposed strategy stems from the idea of injecting faults during the seed reveal phase using a reference function in a tree structure. Mondal et al.[30] explored how faulting specific instructions can lead to leakage or unintended revelation of multiple seeds. However, these schemes do not implement a reference tree mechanism for seed revealing. Their batched GGM structure does not rely on references function, which makes this form of instruction glitching irrelevant for their implementations.

Counter Skipping in AES The work of Jendral et al. [25] introduced a *counter increment skipping attack* against schemes that use AES in counter mode, with the aim of producing duplicate AES output. In our context, AES counter mode is used exclusively in FAEST to derive node values in the GGM tree. In the other Round 2 submissions, by contrast, AES in counter mode is employed only within the secret share expansion routine. During this seed expansion, the required secret share value is generated from a seed of the required length, so duplicating AES output via a counter-skip does not generally yield new share material in those schemes. Consequently, the attack described by Jendral et al. is only practically relevant for FAEST and not for the other examined schemes. Moreover, we expect that a counter-skip during seed expansion would reveal little or no additional information about the secret shares in the non-FAEST implementations.

3.5 Countermeasures

As a countermeasure against successful fault attacks on the seed expansion in the GGM tree, we propose several mitigation strategies. The first two attack vectors, which target the functions `mirath_ggm_tree_expand` and `mirath_expand_seed`, are only effective because the tree data structure is initialized with zeros. To prevent the recovery of the secret key through such faults, the tree should not be initilized with zeros and should contain random values. In addition, checks should be performed to ensure that no tree nodes remain equal to zero after initialization.

Furthermore, a general countermeasure against all attacks presented in this section is to include a check that verifies whether sibling nodes in the GGM tree are equal. If such an equality is detected, the signing procedure should abort and output an error instead of producing a signature. This check also mitigates instruction-skip attacks targeting XOR operations. We evaluated the performance impact of this countermeasure and observed that, averaged over 100,000 signatures, the GGM tree computation requires an additional 13,353 clock cycles. For comparison, the Mirath authors report a total signing cost of approximately 5.9 million clock cycles for the `Mirath-1a-fast` parameter set [2]. Consequently, the introduced countermeasure results in an overhead of roughly 0.2 %.

Another effective countermeasure is to validate a signature before it is accepted or used. All fault attacks described in this work result in invalid signatures, and a verification step can therefore detect and reject faulty outputs. For

the `Mirath-1a-fast` parameter set, such a verification requires approximately 3.3 million clock cycles, which corresponds to an overhead of about 56 % compared to the signing cost.

It is also worth noting that the use of a batched tree, as employed for performance optimization during the opening phase, can naturally reduce the probability of a successful fault injection due to the increased structural complexity of the computation. For this reason, we recommend adopting a one-tree optimization approach in practical implementations.

Finally, we emphasize that the fault attacks presented in this work explicitly manipulate the control flow of the program. Consequently, control-flow integrity mechanisms can also serve as an effective line of defense. Moreover, it is conceivable that design-level countermeasures specific to MPCitH-based signature schemes could be developed to address these attacks. We leave the exploration of such design-based mitigations as future work.

4 Forgery Attack via Challenge Manipulation

This section introduces a fault-based forgery strategy that targets the challenge of linear combination generated during the Fiat–Shamir transform in MPCitH-based schemes. The attack specifically focuses on the verification algorithm.

Fault injection attacks targeting the verification phase of digital signature schemes have been explored across various cryptographic families. For lattice-based schemes, for instance, it was demonstrated that instruction skip faults, particularly those bypassing correctness or size validation checks, can lead to the acceptance of invalid signatures [11,14]. Very recently, Schneider et al. showed that even hardened elliptic-curve-based verification routines remain vulnerable to single instruction-skip faults, allowing trivial forgeries in secure boot implementations [32]. These findings confirm that verification algorithms represent a critical attack surface for physical adversaries.

Although skipping verification checks through fault injection represents a straightforward attack vector, it is typically well protected in practical implementations [14]. In contrast, our approach provides an alternative by targeting the internal computation of the Fiat–Shamir challenge while maintaining the normal verification flow. This strategy preserves the appearance of legitimate operation while fundamentally subverting the verification of the underlying hard mathematical problem.

Building on practical fault attacks against SHAKE-256 [27], we demonstrate how controlled challenge manipulation enables a forged signature to pass full verification. Although theoretical vulnerabilities in Fiat–Shamir are known [28], our work provides a practical realization of such attacks through fault injection, effectively bypassing parameter-based mitigations.

4.1 Attacker Model

The attacker's goal is to achieve acceptance of forged signatures by the verification algorithm. We consider an attacker with physical access to the verification

device who can induce faults during the computation of the Fiat-Shamir challenge $\mathbf{\Gamma}$. Specifically, the attacker targets the SHAKE function to fix the challenge to a predetermined value.

This model uses practical fault injection capabilities to compromise the cryptographic verification process. We demonstrate how this targeted fault enables effective signature forgery across multiple MPCitH-based schemes while preserving the external appearance of normal verification execution.

4.2 Attack Strategy

The adversary targets the first Fiat–Shamir challenge $\mathbf{\Gamma}$ and injects a fault that forces it to equal a fixed, known output. This manipulation breaks the soundness of the protocol by collapsing the verification condition, thereby enabling forgery of signatures.

To demonstrate the feasibility of the $\mathbf{\Gamma}$-manipulation attack, we first describe in detail how to apply it to RYDE and Mirath, two MPCitH-based schemes with similar transcript structures and verification conditions. We concretely demonstrate how to fix $\mathbf{\Gamma}$ in verification, resulting in a successful signature forgery for arbitrary messages.

Forgery in RYDE and Mirath Both RYDE and Mirath derive their first challenge $\mathbf{\Gamma}$ from a hash function via the Fiat Shamir transform. This challenge is central to verifying the correctness of the witness, as follows: In RYDE, the verifier checks $P_\alpha(z) = \mathbf{v}_{\mathtt{eval}} + \mathbf{\Gamma}\big(\mathbf{H} \cdot \mathbf{x}_{\mathtt{eval}} - \mathbf{y}z^2\big)$, and in Mirath, verifier checks $P_\alpha(z) = \mathbf{v}_{\mathtt{eval}} + \mathbf{\Gamma}\big(\mathbf{H} \cdot \mathrm{vec}(\mathbf{E}_{\mathtt{eval}}) - \mathbf{y}z^2\big)$. In both cases, the vectors $\mathbf{x}_{\mathtt{eval}}$ and $\mathbf{E}_{\mathtt{eval}}$ are derived from opened polynomial commitments, z is a random eveluation point, and $\mathbf{\Gamma}$ serves as a matrix challenge applied to the RSD problem to be verified.

The goal is to produce a forged signature σ^* for any chosen message $\mathtt{msg}^*$ that will be accepted by the faulted verification process. The attack relies on fixing $\mathbf{\Gamma}$ to an attacker-chosen value $\mathbf{\Gamma}^*$. This allows the attacker to compute a fake witness that causes all verification equations to hold, even though the underlying structure has not been satisfied.

The attack is executed in two phases, as explained in the following.

1. Forgery preparation:

1. Choose the fixed output $\mathbf{\Gamma}^* \in \mathbb{F}_{q^m}^{(n-k)\times\rho}$, that will be forced during verification.
2. Compute a fake witness:
   Zero $\mathbf{\Gamma}^*$: If $\mathbf{\Gamma}^*$ were the all-zero matrix, then $P_\alpha(X) = P_{\mathbf{v}}(X)$ in both protocols. The verification becomes $P_\alpha(z) = \mathbf{v}_{\mathtt{eval}}$, which holds automatically because $P_{\mathbf{v}}(z) = \mathbf{v}_{\mathtt{eval}}$. In this case, no condition on $\mathbf{x}_{\mathtt{eval}}$ or $\mathbf{E}_{\mathtt{eval}}$ is ever tested; the witness $\mathbf{x}$ or $\mathbf{E}$ could be arbitrary, and the check passes. Hence, a forger who manages to manipulate the Fiat–Shamir challenge $\mathbf{\Gamma}$ to be zero

could produce a signature that is accepted during verification with a fault, without solving the underlying rank problem.
General $\mathbf{\Gamma}^*$: Generally, any $\mathbf{\Gamma}$ that makes the product $\mathbf{\Gamma} \cdot (\mathbf{H}(P_{\mathbf{x}}(z)) - \mathbf{y}z^2)$ the degree one polynomial will trivialize the proof. The attacker chooses $\mathbf{\Gamma}^*$ and then solves for a witness $(\mathbf{s}', \mathbf{C}')$ such that $\mathbf{\Gamma}^* \cdot \big((1 \parallel \mathbf{s}') \cdot [\mathbf{I}_z \parallel \mathbf{C}'] \cdot \mathbf{H}^\top - \mathbf{y}\big) = \mathbf{0}$. This system should be easy to solve as the elements of a fixed $\mathbf{\Gamma}$ can be used to cancel the remaining terms of the RSD problem for the fake witness.Alternatively, the attacker may fix some entries in $\mathbf{C}'$ and solve the resulting system, which must be in the kernel of $\mathbf{\Gamma}^*$.

3. Simulate signature generation: To simulate a valid-looking signature, the attacker takes the following steps:
   (a) For each iteration $e \in \{1, \ldots, \tau\}$, generate the shares and use the forged witnesses to compute $\mathbf{s}'^{(e)}_{\mathtt{aux}}$ and $\mathbf{C}'^{(e)}_{\mathtt{aux}}$, then commit.
   (b) For each iteration $e \in \{1, \ldots, \tau\}$, generate polynomials $P_{\mathbf{s}'}(X) = \mathbf{s}' \cdot X + \mathbf{s}'_{\mathtt{base}}, P'_{\mathbf{C}}(X) = \mathbf{C}' \cdot X + \mathbf{C}'_{\mathtt{base}}$ and $P_v(X) = \mathbf{v} \cdot X + \mathbf{v}_{\mathtt{base}}$
   (c) For each iteration $e \in \{1, \ldots, \tau\}$, compute $P_x^{(e)}(X)$ by computing

$$x^{(e)}_{\mathtt{mid}} = \left[\mathbf{s}^{(e)}_{\mathtt{base}} || \mathbf{s}^{(e)}_{\mathtt{base}} \cdot \mathbf{C}'^{(e)}_{\mathtt{aux}} + \mathbf{s}^{(e)} \cdot \mathbf{C}'^{(e)}_{\mathtt{base}}\right]$$

   and

$$x^{(e)}_{\mathtt{base}} = \left[\mathbf{0}_{m \times r} || \mathbf{s}^{(e)}_{\mathtt{base}} \cdot \mathbf{C}'^{(e)}_{\mathtt{base}}\right],$$

   where $\mathbf{s}^{(e)}_{\mathtt{base}} = (1 \parallel \mathbf{s}'^{(e)}_{\mathtt{base}})$.
   Compute the polynomial $P^{(e)}_{\alpha} = \alpha^{(e)}_{\mathtt{mid}} \cdot X + \alpha^{(e)}_{\mathtt{base}}$ by computing

$$\alpha^{(e)}_{\mathtt{mid}} = \mathbf{x}^{(e)}_{\mathtt{mid}} \cdot (\mathbf{I}_{m \cdot (n-k)} \parallel \mathbf{H}')^\top \cdot \mathbf{\Gamma}^* + \mathbf{v}^{(e)}$$

   and

$$\alpha^{(e)}_{\mathtt{base}} = \mathbf{x}^{(e)}_{\mathtt{base}} \cdot (\mathbf{I}_{m \cdot (n-k)} \parallel \mathbf{H}')^\top \cdot \mathbf{\Gamma}^* + \mathbf{v}^{(e)}_{\mathtt{base}}$$

   (d) Proceed with the simulation by computing

$$h_2 = \mathtt{Hash}_2\left(\mathtt{Hash}_0(\mathtt{msg}), \mathtt{pk}, \mathtt{salt}, h_1, \left(\alpha^{(e)}_{\mathtt{mid}}, \alpha^{(e)}_{\mathtt{base}}\right)_{e \in [1,\ldots,\tau]}\right)$$

   and compute the open random evaluation to generate the signature

$$\sigma^* = \left(\mathtt{salt} \,|\, \mathtt{ctr} \,|\, h_2 \,|\, \pi_{\mathtt{BAVC}} \,|\, \left(\mathbf{s}'^{(e)}_{\mathtt{aux}}, \mathbf{C}'^{(e)}_{\mathtt{aux}}, \alpha^{(e)}_{\mathtt{mid}}\right)_{e \in [1,\ldots,\tau]}\right).$$

This signature is not valid under honest verification, but will become valid once $\mathbf{\Gamma}^*$ is forced during verification with a fault.

2. Fault injection during verification:

During verification, the verifier computes $\mathbf{\Gamma} \leftarrow \mathtt{ExpandChallenge1}(h_1)$. The attacker injects a fault to force $\mathbf{\Gamma} = \mathbf{\Gamma}^*$, the same value used to generate the fake witness.

Verification with $\mathbf{\Gamma}^*$ should hold from construction by checking the steps below:

1. Computation check:

$$\mathbf{x}_{\text{eval}}^{(e)} = \left[\mathbf{s}_{\text{eval}}^{(e)} \cdot \phi(i^{*(e)}) \,\middle\|\, \mathbf{s}_{\text{eval}}^{(e)} \cdot \mathbf{C}_{\text{eval}}^{\prime(e)}\right] \quad \texttt{and}$$
$$\alpha_{\texttt{eval}}^{(e)} = \mathbf{v}_{\texttt{eval}} + \left(\mathbf{x}_{\texttt{eval}}^{(e)} \cdot \left(\mathbf{I}_{m\cdot(n-k)} \,\|\, \mathbf{H}'\right)^{\top} - \mathbf{y} \cdot \phi(i^{*(e)})^2\right) \cdot \mathbf{\Gamma}^*.$$

2. Verifying $h_2' = h_2$ and $v_{\texttt{grinding}} = 0$. Thus, the forged signature is accepted.

4.3 Practical Attack

In this section, we discuss concrete attack vectors from the reference implementations of the various round 2 MPCitH submissions to explain for the attack described above how it can be realized in practice.

Mirath In Mirath, the challenge matrix $\mathbf{\Gamma}$ is generated by hashing the value `h_sh` with SHAKE. Since version 2.1 of the implementation, two other functions are called for this purpose: first `seedexpander_shake_init` (see Listing 1.2) is executed to initialize hash state, and then, to extract the output bytes into the variable `Gamma`, `seedexpander_shake_get_bytes` is used. The initialization routine `seedexpander_shake_init` first initializes SHAKE with `Keccak_HashInitialize_SHAKE128`, then injects the value of `h_sh` via `Keccak_HashUpdate`. If a salt is provided it would also be absorbed, but in the computation of $\mathbf{\Gamma}$ no salt is passed. The absorption phase is completed by a call to `Keccak_HashFinal`.

```
1 void seedexpander_shake_init(shake_prng_t *
      seedexpander_shake, const uint8_t* seed, size_t
      seed_size, const uint8_t* salt, size_t salt_size) {
2     Keccak_HashInitialize_SHAKE128(seedexpander_shake);
3     Keccak_HashUpdate(seedexpander_shake, seed,
          seed_size << 3);
4     if(salt != NULL) Keccak_HashUpdate(
          seedexpander_shake, salt, salt_size << 3);
5     Keccak_HashFinal(seedexpander_shake, NULL);
6 }
```

Listing 1.2. Initialization of SHAKE in Mirath v2.1.

To produce a predictable challenge, an attacker can skip the call to `Keccak_HashUpdate` so that no input is absorbed into the SHAKE state. In that case SHAKE outputs the digest of an empty input, which is a fixed and known value.

RYDE In RYDE, the matrix $\mathbf{\Gamma}$ is generated using the `ryde_1f_tcith_expand_challenge_1` function, which takes both the hash value `h1` and a salt as input. Since the matrix $\mathbf{\Gamma}$ is initialized with zero entries, skipping this function call results in a challenge consisting entirely of zeros. This makes the function a

viable target for fault injection attacks. The function `ryde_1f_tcith_expand_challenge_1` is defined in the reference implementation file `tcith.c`, starting at line 159.

Another attack vector in RYDE is the `seedexpander_shake_get_bytes` in the `ryde_1f_tcith_expand_challenge_1` function. Within the SHAKE function, the variable `random` is used as the target variable. The reason for this is that `random` has the correct file format for the SHAKE results. To write the variable `random` to the matrix $\mathbf{\Gamma}$, the function `rbc_53_mat_from_string` is called. If this function call is skipped, the values of $\mathbf{\Gamma}$ remain at the initialized state in this case as well.

In [27], a successful attack on SHAKE was presented in which the absorption phase was skipped using an instruction skip. In the SHAKE implementation of RYDE, this is not possible with a single instruction skip, as the `Keccak_HashUpdate` function is initialized with a salt value in addition to the hash value `h1`. By skipping `Keccak_HashUpdate` for the input data, the same function call remains for the salt value.

Furthermore, the function `seedexpander_shake_init` cannot be skipped, because otherwise the pointer `seedexpander` is not initialized and the program would end up in a segmentation fault.

4.4 Countermeasures

In this subsection, we discuss potential countermeasures for forgery attacks targeting challenge generation. We focus on mitigation strategies specifically for those schemes in which we have identified a viable attack vector. As already discussed in Section 3.5, the attacks presented here can be effectively prevented by deploying appropriate control-flow protection mechanisms, which ensure the correct execution of security-critical functions and make instruction-skipping faults detectable.

Mirath In Mirath, the attack becomes feasible only if the SHAKE function is executed without any input. Since SHAKE produces deterministic output, the resulting challenge matrix $\mathbf{\Gamma}$ will always be the same and can be precomputed. As a countermeasure, we recommend verifying during signature verification whether the generated $\mathbf{\Gamma}$ matrix matches this precomputed value. If such a match is detected, the verification process should be aborted.

We note, however, that this countermeasure itself may be vulnerable to a fault injection. If an attacker manages to bypass the check through another fault, the mitigation could be made ineffective. Such second-order fault attacks habe been practically demonstrated against other cryptographic schemes [12]. Therefore, implementing this countermeasure in a fault-resistant way, such as using redundant or hardened checks, is essential for maintaining security.

RYDE The first attack on RYDE is effective because the challenge matrix $\mathbf{\Gamma}$ is initially filled with zeros. To prevent this, the matrix should not be initialized

with zero values. The second attack exploits the fact that the entire computation is performed on a serialized string representation, and the result is only converted into a matrix at the end. As a countermeasure, a comparison should be performed before and after the computation. If the resulting matrix still contains only the initial values, the verification should be aborted.

5 Applicability to Other Schemes

In this section we evaluate the applicability of the attacks presented in Section 3 and Section 4 to the remaining MPCitH signature schemes. We first deal with the key-reveal attack and then with the signature-forgery attack. For each scheme, where applicable, we present both an attack strategy and a practical attack.

5.1 Key-Recovery Attack on GGM Tree Seed Generation

Attack Strategy for SDitH and FAEST The attack strategy for these signature schemes is very similar to the approaches used for Mirath and RYDE, which were discussed in Section 3.2. For SDitH and FAEST, the secret shares of an instance are summed and combined with the secret to produce an auxiliary value that is included in the signature. Consequently, if an attacker recovers all secret shares for a particular instance, they can calculate the sum of the secret shares and thereby recover the secret itself by combining the sum of the secret shares with the publicly known auxiliary value.

Practicality for SDitH In SDitH, we found no practical attack vectors targeting tree expansion or seed expansion for key recovery, although such attacks remain feasible in theory; the XOR vector, however, is effective. The SDitH signature scheme constructs a GGM tree similar to the schemes Mirath and RYDE. Starting from a randomly chosen root node, the left and right child nodes are derived by executing an AES encryption on a defined message.

SDitH makes use of structured data types to manage variables, including those used for representing GGM trees. The tree-related structures are initialized through functions such as `ggm_multi_sibling_tree_init` (defined in `ggm.c`, line 457), which primarily set parameters and configuration values. However, the actual node values of the tree are neither initialized nor generated at this stage. Instead, node values are created on demand. As a result, there is no default initialization of the nodes, and attacks targeting the tree expansion (e.g., via `ggm_multi_sibling_tree_get_leaf_seed_commit`, defined in `ggm.c`, line 607) cannot be used to recover the secret key. The seed generation process cannot be skipped either, as all nodes are derived dynamically as required.

Depending on the parameter set and SDitH version, seed generation invokes AES-based functions, such as `aes128_ctrle_oneshot_encrypt_2blocks_ref` (defined in `aes128_ctrle_special_ref.c`, line 25). In this function, the seeds for a pair of sibling nodes are computed, with the counter value `ctr.v8` incremented between the two encryption operations. If the instruction responsible for

incrementing this counter is skipped, both sibling nodes receive the same seed value, making a key-recovery attack feasible.

Practicality for FAEST In FAEST, both the tree-expansion and seed-expansion vectors are exploitable for key recovery, while the XOR attack vector is not applicable. A GGM tree is used to generate the secret shares. The node values for the tree are produced in the function `generate_seeds`, and individual sibling seeds are derived via calls to `expand_seeds`. Unlike most MPC-in-the-Head signature schemes, FAEST does not distinguish sibling nodes by XORing a single bit into the message before AES; instead, AES is used in counter mode to derive the two siblings. Consequently, the XOR-based attack vector is not applicable to FAEST. The nodes are initialized with the following call:

```
nodes = calloc(2 * params->faest_param.L - 1, lambda_bytes);
```

Because `calloc` zero-initializes the allocated memory, the initial values of the nodes are all zero. This enables both the expand-tree attack and the expand-seed attack as potential vectors for key recovery.

Attack Strategy for MQOM For MQOM, it must be considered that the auxiliary value included in the signature cannot be directly used to reconstruct the secret from the secret shares. The reason for this is that the first λ bits of the auxiliary value are truncated. Nevertheless, the secret can still be derived from the auxiliary value. We consider the following equation for the untruncated auxiliary value $\Delta_x[e]$:

$$\Delta_x[e] = x - \sum_{i=0}^{N-1} \bar{x_i} \,.$$

The first λ bits of x are defined as δ. The parameter λ is used during the generation of the seed nodes: when creating sibling nodes, the right child node is computed as the value of the left child node plus δ. When all secret shares $\bar{x_i}$ are summed, the first λ bits of the result correspond to δ. Consequently, when the sum of the secret shares is subtracted from the secret x, the first λ bits of the result become zero. Therefore, even though the first λ bits are truncated in the auxiliary value, the secret can still be reconstructed from $\Delta_x[e]$ by defining the first λ bits as zero.

Practicality for MQOM For MQOM, we were unable to identify any attack vector that yields a reliable key recovery in practice. In the scheme, the first seed for the root node is set using the random value `rseed`. All subsequent nodes are derived through a `SeedDerive` function. MQOM employs the *correlated tree optimization*, in which only one value of a sibling pair is generated using a PRG, while the second is computed via an XOR operation.

For the commitment phase, the function `BLC_Commit`, which is part of the reference implementation and defined in the file `blc.c` starting at line 39, is called. This function internally generates τ trees using the `GGMTree_Expand` function. Skipping the `GGMTree_Expand` function is not a reliable fault attack strategy. The corresponding node values (`lseed`) are not initialized and may contain leftover memory values. An attacker who has knowledge of these memory contents might be able to carry out a successful attack. However, in the general case, the resulting node values are random and cannot be linked to any meaningful state, making the attack infeasible. A similar situation occurs if the `SeedDerive_x2` function is skipped. Because the nodes are not initialized, they hold arbitrary and unpredictable memory content, making it impossible for an attacker to reliably control the outcome of a fault and exploit it. Unlike schemes such as Mirath, RYDE, and SDitH, MQOM does not derive sibling nodes using the same message XORed with 1. Instead, different messages are used for each sibling. Therefore, skipping the XOR instruction is not a viable attack vector in MQOM.

Attack Strategy for PERK The PERK specification [1] details a signing process where the secret witness is embedded into VOLE correlations derived from a GGM tree. If an adversary compromises all seeds in this tree, they can recover the signer's long-term secret key by inverting this embedding process.

1. Reconstruct VOLE secrets: Using all seeds $\{\mathsf{seed}_{e,i}\}$ and the public salt, the adversary executes $\mathsf{ConvertToVOLE}$ (Alg. 4.4) and $\mathsf{VOLECommit}$ (Alg. 4.5) to recover the prover's VOLE secret $\mathbf{u}$.
2. Extract compressed witness: From the signature component $\mathbf{t}$ and the reconstructed $\mathbf{u}$, the adversary computes:

$$\mathbf{w} = \mathbf{t} \oplus \mathbf{u}_{[0:\ell-1]}$$

   This reverses the witness masking step in $\mathsf{P.VOLE\text{-}ElementaryVector}$ (Alg. 4.17, line 2) and $\mathsf{PERK.Sign}$ (Alg. 4.34, line 12).
3. Decode the permutation: The compressed witness $\mathbf{w}$ is a concatenation of blocks $\mathbf{w}'_i$. Each block is decoded by inverting the $\mathsf{PosToWitness}$ encoding (Alg. 4.12), which implements a bijective mapping from a position pos_i to a bit string. The inverse mapping parses the elementary vectors in $\mathbf{w}'_i$ to recover the non-zero position pos_i for each row i of the secret permutation matrix $\mathbf{P}$.
4. Reconstruct the secret key: The set of positions $(\mathsf{pos}_0, \ldots, \mathsf{pos}_{n-1})$ defines the permutation matrix $\mathbf{P}$, which constitutes the functional secret key.

Practicality for PERK In PERK, every attack vector analyzed can be used to perform key recovery. The node values of the GGM tree are stored in the variable `big_tree`, which is initialized to zero. The tree values are computed by the function `expand_ggm_tree`, and the child nodes are generated inside `ggm_expand_seed`. Therefore, both the expand tree attack vector and the expand

seed attack vector are applicable to PERK and can lead to key recovery when exploited.

Since version 2.1 PERK also offers an AES-based mode in which node values are derived using AES. As in Mirath and RYDE, the only difference between the two sibling messages is an XOR operation. Therefore, in the AES mode the XOR attack is also feasible.

5.2 Forgery Attack via Challenge Manipulation

Attack Strategy for MQOM and SDitH The above attack applies in a similar manner to MQOM and SDitH. In the MQOM specification[9, Section 2.1.2], the security of the 5-round variant relies critically on the linear combination challenge matrix $\mathbf{\Gamma}$ used to compute $P_\alpha = P_u + \mathbf{\Gamma} \cdot \hat{F}(P_x)$. If an adversary could fix $\mathbf{\Gamma}$ to zero or another predetermined value, they could completely bypass the multivariate quadratic hardness assumption. Specifically, setting $\mathbf{\Gamma} = 0$ would reduce the equation to $P_\alpha = P_u$, effectively eliminating the term that encodes the MQ problem instance. This would allow forgery without knowledge of a valid secret x, as the verification check would degenerate to verifying only the random masking polynomial P_u.

For SDitH, a similar critical dependency exists on the batching challenge $\gamma_1, \ldots, \gamma_m$ used in the linear combinations of instances

$$P_\alpha(X) = P_0(X) \cdot X + \sum_{j=1}^{m} \gamma_j \cdot f_j(P_1(X), \ldots, P_m(X))$$

If an adversary could fix all $\gamma_j = 0$, then $P_\alpha(X) = P_0(X) \cdot X$, completely removing the witness-dependent constraints f_j. This would allow forgery without knowledge of a valid witness, as verification would only check the random masking polynomial P_0. The Fiat-Shamir derivation of γ_j from the commitment h_{lines} is therefore essential to prevent this attack. Additionally, the consistency check matrix M expanded from h_{aux} via `ExpandConsistencyChallenge` ensures that the τ committed polynomials encode the same witness. If M could be fixed, the consistency check would be bypassed, allowing inconsistent witness encodings to pass verification. Both the TCitH and VOLEitH frameworks used in SDitH variants rely on the unpredictability of these challenges for soundness, making them susceptible to similar attacks if challenge derivation is compromised.

Practicality for SDitH In SDitH, we did not identify any practical attack vector enabling a signature forgery, although such an attack appears theoretically possible. As part of the verification process, the consistency check matrix is generated via the function `both_vole_consistency_check_matrix`, where the variable `cchk_matrix_rng` is defined. This function can be found in the round 2 reference implementation, specifically in `vole_expansion.c`, starting from line 193. This variable represents the challenge used in the check. As in other parts of SDitH, the SHAKE function is used to generate the challenge. To make the

SHAKE output independent of the actual input, an attacker might attempt to skip the call to `xof_finalize_and_output`. However, since `cchk_matrix_rng` is not initialized before use, the resulting output cannot be predicted or controlled. As a result, skipping the finalization does not provide any meaningful advantage for an attacker.

Practicality for MQOM In MQOM, we successfully identified an attack vector that enables a signature forgery. Two different approaches are used to construct the challenge matrix $\mathbf{\Gamma}$, depending on whether a batched tree variant is employed. In the case without a batched tree, the attack cannot be carried out because we do not receive any additional information from the challenge polynomial.

In contrast, when using the batched tree variant, the challenge matrix $\mathbf{\Gamma}$ is generated via SHAKE over the commitment value. This process takes place within the `ComputePAlpha` function (defined in `piop.c`, starting at line 69), where the SHAKE function is executed in three steps: `xof_init`, `xof_update`, and `xof_squeeze`. During one of the `xof_update` calls, the commitment `com` is passed as input to the hash function. If this specific update call is skipped due to an instruction skip, the resulting hash value becomes constant and independent of the commitment. This behavior enables a forgery attack in the batched tree variant, as the generated challenge matrix is no longer related to the actual commitment.

Attack Strategy for PERK The PERK signature scheme implements a zero-knowledge proof of knowledge for the Permuted Kernel Problem (PKP) within the VOLE-in-the-Head (VOLEitH) framework. The PKP instance is defined by a public matrix $\mathbf{H} \in \mathbb{F}_q^{m \times n}$ and vector $\mathbf{x} \in \mathbb{F}_q^n$, where the prover demonstrates knowledge of a permutation matrix $\mathbf{P} \in \mathbb{F}_q^{n \times n}$ satisfying

$$\mathbf{HPx} = \mathbf{0}.$$

The scheme exhibits a critical vulnerability to forgery attacks when the Fiat-Shamir challenge responsible for constraint combination can be fixed. This challenge, denoted ch_2 in the specification and derived from the hash function H_2^2 instantiated with SHAKE, generates the random coefficients $\alpha \in \mathbb{F}_{2^\rho}^{cn+n+m}$ that linearly combine all proof constraints, elementary vector structure, column summation, and PKP satisfiability into a single master polynomial $f(X)$. The soundness of this construction relies on the unpredictability of α to ensure that any invalid witness produces a non-zero leading coefficient in $f(X)$ with overwhelming probability over the choice of α.

However, if an adversary can predetermine ch_2 through fault injection, they can strategically compute a malicious linear combination that systematically cancels the leading coefficient of $f(X)$ for an arbitrary invalid witness. This manipulation bypasses the core soundness check in $\mathsf{V.Check\text{-}PKP}$ by ensuring the masked polynomial appears valid despite being generated from a fraudulent permutation matrix $\mathbf{P}^*$ that does not satisfy $\mathbf{HP}^*\mathbf{x} = \mathbf{0}$. Consequently, an attacker

can forge signatures for arbitrary message-public key pairs without solving the underlying PKP instance, completely violating the scheme's existential unforgeability guarantees.

Practicality for PERK In PERK, we identified an attack vector that allows the execution of a successful signature forgery. As outlined in the attack strategy, a signature forgery against PERK can be performed if the second challenge ch_2 assumes a constant value. In the PERK v2.1 implementation this challenge is initialized to zero. The value of the variable `ch_2` is derived by the function `sig_perk_gen_second_challenge`, which computes the hash value used for the challenge. If an instruction skip causes this function to be bypassed, `ch_2` remains zero. Therefore, a successful fault injection that skips `sig_perk_gen_second_challenge` enables a practical signature forgery against PERK.

Attack Strategy for FAEST Within the FAEST signature scheme's VOLE-in-the-Head framework, the zero-knowledge proof phase relies on a critical challenge for its soundness. This occurs in the QuickSilver protocol, where after a prover has committed to their witness, they must prove that a set of C constraint polynomials $\{p_i(X)\}$ all evaluate to zero. To efficiently verify this, the protocol employs a random linear combination. A challenge vector $\mathbf{r}$ is generated, which is used to compress all constraints into a single, combined polynomial: $P_{\text{combined}}(X) = \sum_{i=0}^{C-1} \mathbf{r}_i \cdot p_i(X)$. The prover must then demonstrate that $P_{\text{combined}}(X)$ is the zero polynomial. The security of this step rests entirely on the unpredictability of $\mathbf{r}$; if even one original constraint is invalid, a randomly chosen $\mathbf{r}$ will cause the combined check to fail with probability nearly 1.

This leads to a devastating forgery attack. If an adversary can subvert the challenge generation; for instance, by forcing a fault in the H_2^2 hash function whose output defines $\mathbf{r}$, they can deliberately set the coefficients of $\mathbf{r}$ to zero. This action systematically breaks the proof's soundness. With $\mathbf{r} = 0$, the combined constraint polynomial becomes zero by construction, regardless of whether the original constraints were satisfied. Consequently, an attacker can forge a signature for any message and public key without any knowledge of the valid secret key, as they can bypass the core proof of knowledge verification entirely.

Practicality for FAEST In FAEST, we did not identify any practical attack vector enabling a signature forgery, although such an attack appears theoretically possible. For a successful signature forgery, it is necessary that all variables satisfy $r_i = 0$. In the implementation these values appear as the variables `a0_tilde`, `a1_tilde`, and `a2_tilde`. These variables are not initialized to zero in the reference code, and therefore there is no immediate or obvious fault-injection vector that would force them all to zero and thus enable the planned signature-forgery attack.

6 Implementation

In this section, we first introduce the experimental setup used for our fault injection attacks. We then demonstrate the feasibility of the attacks presented in this work by successfully executing them on the Mirath and RYDE signature schemes. By applying the described attack techniques to real hardware, we validate their practical relevance and evaluate their success rates.

6.1 Experimental Setup

For the attacks, we use a setup consisting of a ChipWhisperer Lite, a UFO board, and as an target baord STM32L5HWC [15]. To the best of our knowledge, there is no specific implementation for embedded systems for Mirath or RYDE. Therefore, we implement only the required subfunctions for the experiments, because the entire scheme exceeds the memory requirements during runtime. We choose STM32L5HWC as the target, as it has sufficient memory for these subfunctions and also has a hardware AES module. Communication with a PC takes place via the SimpleSerial protocol. This allows inputs for the implemented functions to be sent, as well as outputs to be received and viewed. On the PC side, a Jupyter Notebook instance is used to control the measurement process and analyze the results.

For the attacks presented in this work, we use clock glitches with the goal of inducing instruction skips. Glitches are injected based on triggers placed in the target program. A trigger is a signal sent from the target device to the ChipWhisperer, which allows the attacker program to react with specific actions, such as injecting a fault at a precise point in time. To obtain a setup that is as realistic as possible, the triggers are placed around the implemented function rather than directly at the instruction where the fault is intended to occur. In a realistic attack scenario, the adversary does not have precise knowledge of the program state or instruction timing at any given moment. Consequently, the trigger encloses the entire target function instead of a specific subfunction or instruction. This increases the difficulty of the attack, as the adversary must identify the exact point within the function at which the fault should be injected. Such a point can be determined either through program analysis or by evaluating side-channel measurements. Likewise, in a real attack without triggers, the attacker can learn the start time of the target function. For all experiments, we use the clock glitch parameters listed in Listing 1.3.

```
scope.glitch.clk_src = "clkgen"
scope.glitch.output = "clock_xor"
scope.glitch.trigger_src = "ext_single"
scope.io.hs2 = "glitch"
```

Listing 1.3. General parameters for the clock glitch attack.

In addition to these settings, Chipwhisperer also has the `offset`, `width` and `ext_offset` parameters to adjust the glitch. We use fixed values for `width` and

`offset`, which have proven to be reliable for an instruction skip. To identify a suitable parameter set of `offset` and `width`, we implemented a sample program on the target device and performed a brute-force search over the parameters on this program. The parameter `ext_offset` is used to place the glitch relative to the trigger set. The setting of `ext_offset` is therefore dependent on the respective attack.

6.2 Key Recovery Attack on GGM Tree Seed Generation

For this attack, we implement the subfunction `mirath_ggm_tree_expand` from the round 2 submission of Mirath. The function is the same in both Mirath and RYDE. The seed tree creation uses a salt as input, which is selected each time it is executed. So in order for this salt to be generated, we create a random 32 byte value in the Jupyter Notebook instance, which is passed as input for each execution.

The target of the attack is a GGM tree instantiated with the parameters from the `Mirath1a-fast` version. For a binary tree, $2N - 1$ nodes are required, where $N = 4352$ corresponds to the number of leaf nodes. Since the root node is already given and each seed expansion generates two child nodes from a parent node, the function `mirath_expand_seed` is called 4351 times to complete the tree. Within this function, two child nodes are derived from a parent seed using AES. To accelerate the computation, we use the hardware AES module available on the STM32L5HWC target board instead of the software-based AES implementation used in the reference code.

Instruction Skip on `mirath_ggm_tree_expand` Function Call For the key-recovery attacks, we use the function `mirath_ggm_tree_expand` as the core of our implementation. Consequently, this function is the first one called in our partial implementation. To model a realistic attack scenario, the trigger is placed around the entire partial implementation, which causes the call to `mirath_ggm_tree_expand` to occur very close to the trigger point. In our experiments we observed that when `mirath_ggm_tree_expand` is the very first function executed after the trigger, the attack does not always succeed. However, by inserting a `memcpy` operation—as present in the reference implementation—between the trigger and the GGM tree function, we achieved a 100 % success rate with the glitch parameters given in Listing 1.4, corresponding to an expected number of one trial. The insertion of this additional instruction increases the attack window for the fault injection. If no instruction is executed between the trigger and the function branch, parts of the subsequent instructions may already be loaded into the processor pipeline, which can reduce the reliability of the fault and lead to unsuccessful attacks.

```
scope.glitch.offset = -46.6
scope.glitch.width = -5.5
scope.glitch.ext_offset = 13
```

Listing 1.4. Parameters for an instruction skip on the `mirath_ggm_tree_expand` function call.

Instruction Skip on `mirath_expand_seed` Function Call To demonstrate the practical feasibility of this attack, we target the `mirath_expand_seed` function from Listing 1.5 and show how to skip its execution. When this function is skipped, the two child nodes that it is supposed to generate remain at their default initialized value of zero.

In the reference implementation, the generation of the GGM tree leaves is performed within a loop structure. As an example, we consider the case where the seed creation for $i = 1$ is skipped. As a result, the corresponding child nodes, labeled as nodes 3 and 4, are not computed correctly and retain the zero value. This creates a fault in the tree structure that can be exploited in the context of key recovery.

```
for (size_t i = 0; i < (MIRATH_PARAM_TREE_LEAVES - 1);
    i++) {
    mirath_expand_seed(ggm_tree +
        mirath_ggm_tree_child_0(i), salt, i, ggm_tree[i
        ]);
}
```

Listing 1.5. GGM tree creation in Mirath.

We execute the attack from a Jupyter Notebook instance. Here, we also create the random 32-byte salt value and transfer it to the target program via SimpleSerial. We use the `GlitchController` class to record the results. It distinguishes between the events `success`, `reset`, and `normal`. After executing the `mirath_ggm_tree_expand` function, the target program checks whether nodes 3 and 4 of the created tree are the same. If this is the case, the program responds with an "w", otherwise with an "l". The output of the program is then evaluated as `success` or `normal`. If the program no longer responds due to the clock glitch and the target has to be restarted, the `reset` event is tracked.

With an `ext_offset` of 162240, the attack can be executed with a success probability of 100 %, meaning that the fault injection reliably causes the two child nodes to be identical. The reason for this is that the functions used are time-constant, even on random data. To find the correct `ext_offset`, we inserted a loop of NOP instructions at the point to be skipped. During a power consumption measurement, we were able to roughly estimate how many clock cycles had passed. Within the range of the estimate, we could then brute-force the `ext_offset` parameter until we had a successful attack.

As already described in Section 3.3, targeting nodes on the first two levels guarantees a successful key recovery with a probability of 100 %, since these

levels always include nodes that can lead to the generation of hidden nodes. We validated this result experimentally by performing fault injections against nodes on the first two levels. The 100 % success rate therefore reflects our practical attacks on those levels.

XOR Instruction Skip In this attack, we demonstrate the feasibility of skipping the XOR instruction during seed expansion. As discussed in Section 3.3, the highest probability of achieving a key recovery is obtained by targeting a node at level 3 of the GGM tree. Therefore, we attack the seed creation at iteration $i = 7$. The goal is to ensure that nodes 15 and 16 in the GGM tree receive identical values.

This is achieved by skipping the XOR instruction that distinguishes the messages used to generate the two sibling nodes encrypted by AES. Unlike the function call skip, this fault requires a different offset that targets the XOR instruction. The offset could also be roughly estimated with a power consumption measurement and then brute-forced until a successful attack was detected. With an `ext_offset` of 1296660, we were able to achieve a success rate of 100 % that the generated nodes are the same.

6.3 Forgery Attack via Challenge Manipulation

For this attack, we use a instruction skip to ensure that the challenge used in the target scheme has constant values. To demonstrate the feasibility of the attack, we present two different attacks on the schemes Mirath and RYDE. No attack vectors were found for SDitH and PERK.

Mirath To demonstrate the practical feasibility of the forgery attack, we implemented the functions required to compute the challenge matrix $\mathbf{\Gamma}$, namely `seedexpander_shake_init` and `seedexpander_shake_get_bytes`. As previously described in Section 4.3, skipping the branch to the function `Keccak_HashUpdate` inside `seedexpander_shake_init` causes SHAKE to produce the hash value of an empty input. To verify whether the fault injection was successful, we check whether the output of the affected function matches this known empty-input SHAKE output. In our experiments we identified reliable glitch parameters that always skip the `Keccak_HashUpdate` function: setting `ext_offset` to 577 consistently skip the branch to `Keccak_HashUpdate`. As with earlier experiments, this offset was determined using a combination of power-trace analysis and brute-force offset tuning.

RYDE We have implemented the two functions `hash_sha3_finalize` and `ryde_1f_tcith_expand_challenge_1` from the reference implementation on the board. Two changes had to be made for the code to work. First, explicit x86 assembly code was used for the `rbc_53_elt_get_degree` function, which we converted to ARM assembly. Second, the initialization of the gamma matrix with

the `rbc_53_mat_init` function resulted in an allocation error. We have therefore adapted the function so that the pointers generated by this function only point to aligned memory addresses.

As described in Section 4.3, we have two possible attack vectors for RYDE. The first attack vector is the function `ryde_1f_tcith_expand_challenge_1`, the second attack vector is the function `rbc_53_mat_from_string`. For an instruction skip in `ryde_1f_tcith_expand_challenge_1`, we set the `ext_offset` to 34398 and achieved a successful attack with a probability of 50 %. For an instruction skip in the `rbc_53_mat_from_string` function, setting `ext_offset` to 70699 resulted in a success probability of 86 %. Consequently, the expected number of trials required for a successful attack lies between one and two for this attack scenario. Again, we were able to find the parameters with a power consumption measurement together with a brute force method. We believe that more successful attacks can be realized with better parameters. We leave this for future work.

7 Conclusion

In this work, we investigated the security of MPC-in-the-Head signature schemes under fault attacks. We introduced two practical fault attacks, a key-recovery attack and a signature-forgery attack, and analyzed their applicability to all six MPCitH-based candidates evaluated in round 2 of NIST's digital signature standardization process: FAEST, Mirath, MQOM, PERK, RYDE, and SDitH.

Our analysis demonstrates that all six schemes are vulnerable to at least one of the proposed attacks. The key-recovery attack, which targets the GGM tree seed generation process, was successfully performed on Mirath and RYDE using a ChipWhisperer setup, confirming its practical feasibility. The forgery attack, which manipulates the Fiat–Shamir challenge computation, was also shown to be effective.

To mitigate these threats, we proposed several practical countermeasures, including initializing GGM trees with random nonzero values, verifying the consistency of sibling nodes, and introducing hardened checks during challenge generation. These techniques can significantly reduce the risk of fault-based key recovery and signature forgery.

Our findings underscore the importance of incorporating physical attack resistance into the design and implementation of MPCitH-based signature schemes, particularly as they progress towards standardization.

Acknowledgments This work has been funded by the Deutsche Forschungsgemeinschaft (DFG) - KR 5152/2-1 and SFB 1119 - 236615297, and by the German Federal Ministry of Research, Technology and Space (BMFTR) under the project QUORYPTAN (16KIS2034).

References

1. Aaraj, N., Bettaieb, S., Bidoux, L., Budroni, A., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Kulkarni, M., Mateu, V., Palumbi, M., Perin, L., Rivain, M., Tillich, J., Xagawa, K.: PERK. Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures
2. Adj, G., Aragon, N., Barbero, S., Bardet, M., Bellini, E., Bidoux, L., Chi-Domínguez, J., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M., Rivera-Zamarripa, L., Sanna, C., Tillich, J., Verbel, J., Zweydinger, F.: Mirath (merger of MIRA/MiRitH). Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures
3. Aguilar Melchor, C., Bettaieb, S., Bidoux, L., Feneuil, T., Gaborit, P., Gama, N., Gueron, S., Howe, J., Hülsing, A., Joseph, D., Joux, A., Kulkarni, M., Persichetti, E., Randrianarisoa, T.H., Rivain, M., Yue, D.: SDitH — Syndrome Decoding in the Head. Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures
4. Aragon, N., Bardet, M., Bidoux, L., Chi-Domínguez, J., Dyseryn, V., Feneuil, T., Gaborit, P., Joux, A., Neveu, R., Rivain, M., Tillich, J., Vinçotte, A.: RYDE. Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures
5. Aranha, D.F., Berndt, S., Eisenbarth, T., Seker, O., Takahashi, A., Wilke, L., Zaverucha, G.: Side-channel protections for picnic signatures. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 239–282 (2021)
6. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: Optimizing GGM trees and OWFs for post-quantum signatures. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part I. LNCS, vol. 15484, pp. 463–493. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0875-1_15
7. Baum, C., Braun, L., Beullens, W., Delpech de Saint Guilhem, C., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: FAEST. Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures
8. Baum, C., Braun, L., Delpech de Saint Guilhem, C., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part V. LNCS, vol. 14085, pp. 581–615. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38554-4_19
9. Benadjila, R., Bouillaguet, C., Feneuil, T., Rivain, M.: MQOM — MQ on my Mind. Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures
10. Bidoux, L., Feneuil, T., Gaborit, P., Neveu, R., Rivain, M.: Dual support decomposition in the head: Shorter signatures from rank SD and MinRank. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 38–69. Springer (2024)

11. Bindel, N., Buchmann, J., Krämer, J.: Lattice-based signature schemes and their sensitivity to fault attacks. In: 2016 workshop on fault diagnosis and tolerance in cryptography (FDTC). pp. 63–77. IEEE (2016)
12. Blömer, J., da Silva, R.G., Günther, P., Krämer, J., Seifert, J.: A practical second-order fault attack against a real-world pairing implementation. In: FDTC. pp. 123–136. IEEE Computer Society (2014)
13. Buss, J.F., Frandsen, G.S., Shallit, J.O.: The computational complexity of some problems of linear algebra. Journal of Computer and System Sciences **58**(3), 572–596 (1999)
14. Calle Viera, A., Berzati, A., Heydemann, K.: Fault attacks sensitivity of public parameters in the dilithium verification. In: Bhasin, S., Roche, T. (eds.) Smart Card Research and Advanced Applications. pp. 62–83. Springer Nature Switzerland, Cham (2024)
15. ChipWhisperer: Open source side-channel analysis tools (2020), https://www.newae.com/chipwhisperer
16. Feneuil, T.: The Polynomial-IOP Vision of the Latest MPCitH Frameworks for Signature Schemes. Post-Quantum Algebraic Cryptography - Workshop 2, Institut Henri Poincaré, Paris, France (2024), presentation
17. Feneuil, T., Rivain, M.: Threshold linear secret sharing to the rescue of MPC-in-the-head. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part I. LNCS, vol. 14438, pp. 441–473. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8721-4_14
18. Feneuil, T., Rivain, M.: Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. Journal of Cryptology **38**(3), 1–82 (2025)
19. Feneuil, T., Rivain, M., Warmé-Janville, A.: Masking-friendly post-quantum signatures in the threshold-computation-in-the-head framework. Cryptology ePrint Archive, Paper 2025/520 (2025), https://eprint.iacr.org/2025/520
20. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Odlyzko, A.M. (ed.) Advances in Cryptology — CRYPTO' 86. pp. 186–194. Springer Berlin Heidelberg, Berlin, Heidelberg (1987)
21. Godard, J., Aragon, N., Gaborit, P., Loiseau, A., Maillard, J.: Single trace side-channel attack on the MPC-in-the-Head Framework. In: International Conference on Post-Quantum Cryptography. pp. 267–293. Springer (2025)
22. Goldreich, O., Goldwasser, S., Micali, S.: How to construct random functions. J. ACM **33**(4), 792–807 (Aug 1986). https://doi.org/10.1145/6490.6503, https://doi.org/10.1145/6490.6503
23. Guo, X., Yang, K., Wang, X., Zhang, W., Xie, X., Zhang, J., Liu, Z.: Half-tree: Halving the cost of tree expansion in COT and DPF. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part I. LNCS, vol. 14004, pp. 330–362. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30545-0_12
24. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from secure multiparty computation. In: Proceedings of the thirty-ninth annual ACM symposium on Theory of computing. pp. 21–30 (2007)
25. Jendral, S., Dubrova, E.: Side-channel and fault injection attacks on voleith signature schemes: A case study of masked FAEST. IACR Cryptol. ePrint Arch. **378** (2025), https://eprint.iacr.org/2025/378
26. Jendral, S., Dubrova, E.: MAYO key recovery by fixing vinegar seeds. Cryptology ePrint Archive, Paper 2024/1550 (2024). https://doi.org/10.62056/ab0ljbkrz, https://eprint.iacr.org/2024/1550

27. Jendral, S., Mattsson, J.P., Dubrova, E.: A single-trace fault injection attack on hedged module lattice digital signature algorithm (ML-DSA). In: 2024 Workshop on Fault Detection and Tolerance in Cryptography (FDTC). pp. 34–43 (2024). https://doi.org/10.1109/FDTC64268.2024.00013
28. Kales, D., Zaverucha, G.: An attack on some signature schemes constructed from five-pass identification schemes. In: International Conference on Cryptology and Network Security. pp. 3–22. Springer (2020)
29. Katz, J., Kolesnikov, V., Wang, X.: Improved non-interactive zero knowledge with applications to post-quantum signatures. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) ACM CCS 2018. pp. 525–537. ACM Press (Oct 2018). https://doi.org/10.1145/3243734.3243805
30. Mondal, P., Adhikary, S., Kundu, S., Karmakar, A.: ZKFault: Fault attack analysis on zero-knowledge based post-quantum digital signature schemes. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part VIII. LNCS, vol. 15491, pp. 132–167. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0944-4_5
31. Sarde, V., Debande, N.: Differential fault attacks on MQOM, breaking the heart of multivariate evaluation. Cryptology ePrint Archive, Paper 2025/1895 (2025), https://eprint.iacr.org/2025/1895
32. Schneider, K., Auer, L., Wagner, A.: Fault attacks on ecc signature verification. IACR Transactions on Cryptographic Hardware and Embedded Systems **2025**(4), 1010–1052 (2025)